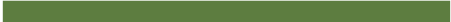




2026 前期期末テスト課題 アンケート 市民意識調査の分析



課題 R6年度市民意識調査

- 自由意見の分類
- 地域別の意見の特徴
- 意見内容の分析
- 要約・キーワードの抽出
- 年代による意見の違い
- 性別による意見のちがい

pdf の一部を取り出す



```
# PDF操作のライブラリをインストール
!pip install pypdf
from pypdf import PdfReader, PdfWriter
```

```
# -----
# 報告書pdfから、必要なページを抜き出す
# -----
```

```
def extract_pages_from_pdf(input_pdf_path, output_pdf_path, target_pages):
```

```
    reader = PdfReader(input_pdf_path)
```

```
    writer = PdfWriter()
```

```
    # ----- 欲しいページを追加 -----
```

```
    for page_num in target_pages:
```

```
        if 1 <= page_num <= len(reader.pages):
```

```
            page = reader.pages[page_num - 1]
```

```
            writer.add_page(page)
```

```
        else:
```

```
            print(f"警告: ページ {page_num} は存在しません。")
```

```
    # ----- 新しいPDFファイルを作成 -----
```

```
    with open(output_pdf_path, "wb") as output_file:
```

```
        writer.write(output_file)
```

```
    print(f"抽出完了: {output_pdf_path} を作成しました。")
```

pdf の一部を取り出す

```
# ----- Main -----
```

```
# pdf fileの読み込み
```

```
path = "/content/drive/MyDrive/...../"
```

```
name = "R6年度市民意識調査.pdf"
```

```
input_file = path + name
```

```
# 保存する新しいPDFファイル名 * * * 注意上書き * * *
```

```
output_name = "調査票.pdf"
```

```
output_file = path + output_name
```

```
# 注意: 実際のPDFに印字されているページ番号ではなく、PDF全体でのページ
```

```
# 例) pages_ = [15, 20] + list(range(40, 46))
```

```
# -----
```

```
# ほしいページの番号を入れる
```

```
# -----es = [n for n in range(226, 238, 1)]
```



```
# 関数の実行
```

```
extract_pages_from_pdf(input_file, output_file, pages)
```



1ページ

2ページ

3ページ

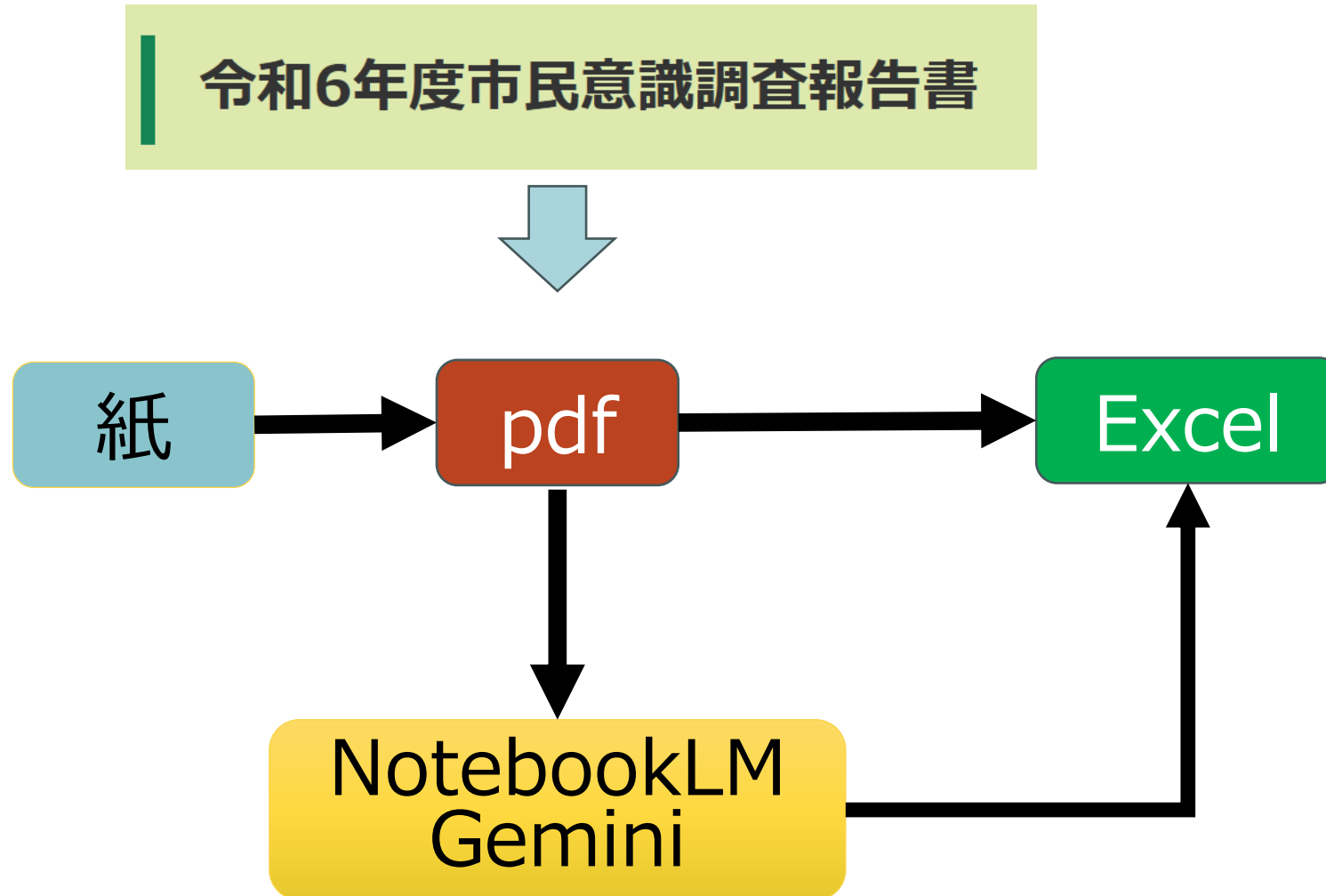
4ページ

5ページ

3ページ

pdf を読み込む

[令和6年度市民意識調査報告書 - 東松山市公式ホームページ](#)



NotebookLM pdf ⇒ csv 変換

The screenshot shows the Gemini NotebookLM interface. The top bar displays the notebook title "Higashimatsuyama Citizen Survey: Visions for Local" and a "Create notebook" button. The interface is divided into three main panels: Sources, Chat, and Studio.

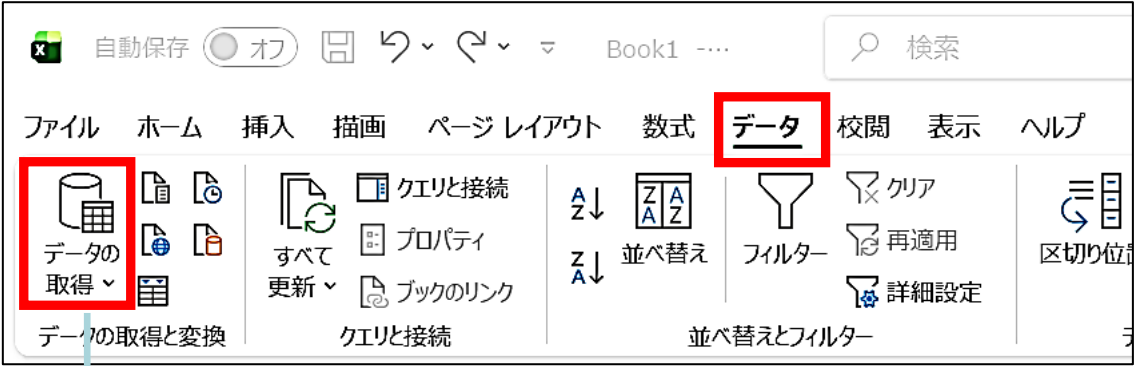
- Sources Panel:** Contains a search bar and a list of sources. A source titled "R6東松山市民ア..." is highlighted with a red box and a checkmark.
- Chat Panel:** Shows a conversation with the user asking for a CSV conversion of a PDF. The AI responds with a confirmation and a request to convert the tables to BOM format CSV.
- Studio Panel:** Displays a list of sources, including "市民意識調査結果..." and "opinion_1_urban_infrastructu...".

A red box highlights the source "R6東松山市民ア..." in the Sources panel, and a blue box highlights the AI's response in the Chat panel.

このCSVファイルは、『R6_市民調査(自由意見).pdf』に含まれる「都市基盤の整備について」や「環境について」、「防災・防犯・安全について」、「保健・医療・福祉について」、「教育・文化・スポーツについて」、「産業について」、「情報について」、「観光について」、「市政について」、「その他」など、全11件のカテゴリーの表データを網羅しています。日本語の文字化けを防ぐため、Microsoft Excel等の表計算ソフトでもそのままきれいに開くことができます。

Gemini NotebookのStudioパネルに自動的に追加されています。

Excel pdf ⇒ csv 変換



R6_市民調査(自由意見).pdf

ファイルから
↓
pdfから



ナビゲーター

Table003 (Page 4)

Column1	Column2	Column3	Column4
性別	年齢	居住地区	市民の声
女性	20～29歳	松山地区	美原町の街灯が少ない夜道怖い
女性	20～29歳	高坂丘陵地区	高坂駅から出るバスの本数が少ない。(特に土日、祝)
男性	30～39歳	野本地区	私の住んでいる地域では、道路に歩道がなく、安心して子どもと歩くことができません。
女性	30～39歳	高坂丘陵地区	小学生が夏休みに遊ぶことのできる施設が極端に少ないと感じます。暑い夏は、公園で遊ぶ機会が減ります。
女性	40～49歳	野本地区	若松町一丁目交差点を改善してほしい。(歩行者分離、矢印信号など信号機を改善してほしい。)
男性	50～59歳	野本地区	市内の渋滞する箇所を解消してほしい。特に南中学校の交差点！
女性	50～59歳	唐子地区	新しく整備されたにも関わらず、危なく感じる交差点や道路があります。改善してほしい。
女性	50～59歳	野本地区	市内バスのバス停を増やしてほしい。
女性	50～59歳	野本地区	近所に新しい戸建て住宅の建築工事がたくさんあり、これからも発展して欲しい。
男性	60～69歳	唐子地区	道路整備や下水道整備について、予算のこともあるでしょうが現実的に困難を感じています。
女性	60～69歳	松山地区	電動カーを使い歩道を利用して買い物をしていますが、道がでこぼこで安定して歩けません。
女性	60～69歳	平野地区	最近市内中心部の一部で歩道の改修工事が行われていましたが、私の住む地域ではまだありません。
男性	70～79歳	野本地区	市役所の庁舎移転について現在の場所は全体的に不便を感じています。将来的に新しい庁舎を建ててほしい。

	A	B	C	D
1	Column1	Column2	Column3	Column4
2	性別	年齢	居住地区	市民の声
3	女性	20～29歳	松山地区	美原町の街灯が少ない夜道怖い
4	女性	20～29歳	高坂丘陵地区	高坂駅から出るバスの本数が少ない。(特に土日、祝、夜23：00頃までの)
5	男性	30～39歳	野本地区	私の住んでいる地域では、道路に歩道がなく、安心して子どもと歩くことができません。
6	女性	30～39歳	高坂丘陵地区	小学生が夏休みに遊ぶことのできる施設が極端に少ないと感じます。暑い夏は、公園で遊ぶ機会が減ります。
7	女性	40～49歳	野本地区	若松町一丁目交差点を改善してほしい。(歩行者分離、矢印信号など信号機を改善してほしい。)
8	男性	50～59歳	野本地区	市内の渋滞する箇所を解消してほしい。特に南中学校の交差点！
9	男性	50～59歳	野本地区	新しく整備されたにも関わらず、危なく感じる交差点や道路があります。改善してほしい。
10	女性	50～59歳	唐子地区	市内バスのバス停を増やしてほしい。
11	女性	50～59歳	野本地区	近所に新しい戸建て住宅の建築工事がたくさんあり、これからも発展して欲しい。
12	女性	50～59歳	野本地区	道路整備や下水道整備について、予算のこともあるでしょうが現実的に困難を感じています。
13	男性	60～69歳	唐子地区	電動カーを使い歩道を利用して買い物をしていますが、道がでこぼこで安定して歩けません。
14	女性	60～69歳	松山地区	最近市内中心部の一部で歩道の改修工事が行われていましたが、私の住む地域ではまだありません。
15	女性	60～69歳	平野地区	市役所の庁舎移転について現在の場所は全体的に不便を感じています。将来的に新しい庁舎を建ててほしい。
16	男性	70～79歳	野本地区	市役所の庁舎移転について現在の場所は全体的に不便を感じています。将来的に新しい庁舎を建ててほしい。

csvファイルをマージして新しいcsvをつくる



```
import glob
import pandas as pd
```

```
# 例: 'data_folder/' フォルダ内にあるすべての .csv ファイル
csv_files = glob.glob("/content/mydrive/MyDrive/.../フォルダー/R6東松山市民アンケートRev(*).csv")
```

```
df_list = [pd.read_csv(file) for file in csv_files]
print(f"合計 {len(csv_files)} 個のファイルを結合(マージ)しました。", "\n")
```

```
# 3. 縦方向(行方向)に結合
# ignore_index=True で結合後のインデックスを振り直します
merged_df = pd.concat(df_list, ignore_index=True)
display(merged_df)
```

```
# 4. 新しいCSVファイルとして保存
path = "/content/mydrive/MyDrive/38_自然言語処理/市民アンケート(2026)/2026/R6東松山市民の意見(all).csv"
```

```
# encoding='utf-8-sig' はExcelで開いた際の文字化けを防ぎます
merged_df.to_csv(path, index=False, encoding="utf-8-sig")
```

```
print(f"----- ファイルを保存しました。 -----")
```


csv ⇒ pandas DataFrameに読み込む

```
# Drive Mount
from google.colab import drive
drive.mount('/content/mydrive')
```

```
import numpy as np
import seaborn as sns
import pandas as pd
from matplotlib import pyplot as plt
!pip install japanize-matplotlib
import japanize_matplotlib
```

```
path = "/content/mydrive/MyDrive/"
file = "R6東松山市民アンケート(all).csv"
df = pd.read_csv(path + file, encoding="UTF-8-sig")
display(df.info())
display(df)
```

	分野	性別	年齢	居住 地区	市民の声
0	都市基盤 の整備に ついて	女性	20~29 歳	松山 地区	美原町の街灯が少ないので夜道が怖い
1	都市基盤 の整備に ついて	女性	20~29 歳	高坂 丘陵 地区	高坂駅から出るバスの本数が少ない。(特に土日、祝、夜23:00頃までの時間) デマンドタクシ...
2	都市基盤 の整備に ついて	男性	30~39 歳	野本 地区	私の住んでいる地域では、道路に歩道がなく、安心して子どもと歩くことができない道路があります。...

```
# 居住地区が全部で何箇所あるか(数値)調べる
```

```
num_areas = df["居住地区"].nunique()
```

```
print(f"居住地区は全部で {num_areas} 箇所です。")
```

```
area_list = df["居住地区"].unique().tolist()
```

```
print(area_list, "¥n")
```

```
category = df["分野"].nunique()
```

```
print(f"分野は{category}種類です。")
```

```
category_list = df["分野"].unique().tolist()
```

```
print(category_list, "¥n")
```

```
age = df["年齢"].nunique()
```

```
print(f"年齢は{age}種類です。")
```

```
age_list = ['18~19歳', '20~29歳', '30~39歳', '40~49歳', '50~59歳', '60~69歳', '70~79歳', '80歳以上']
```

```
print(age_list, "¥n")
```

```
# 居住地区とアンケート回収の数
```

```
# カラムのデータを集計して辞書に変換
```

```
area_dict = df['居住地区'].value_counts().to_dict()
```

```
df_area = pd.DataFrame(list(area_dict.items()), columns=['居住地区', 'アンケート回答数'])  
display(df_area)
```

```
# グラフ作成
```

```
plt.figure(figsize=(7, 5))
```

```
sns.barplot(x="居住地区", y="アンケート回答数", data=df_area, color="blue")
```

```
plt.title("市民アンケート自由意見", fontsize=14)
```

```
plt.xticks(rotation=60, ha="right", fontsize=14)
```

```
plt.xlabel("市内地域別", fontsize=16)
```

```
plt.ylabel("アンケート回答数", fontsize=14)
```

```
plt.yticks(fontsize=14)
```

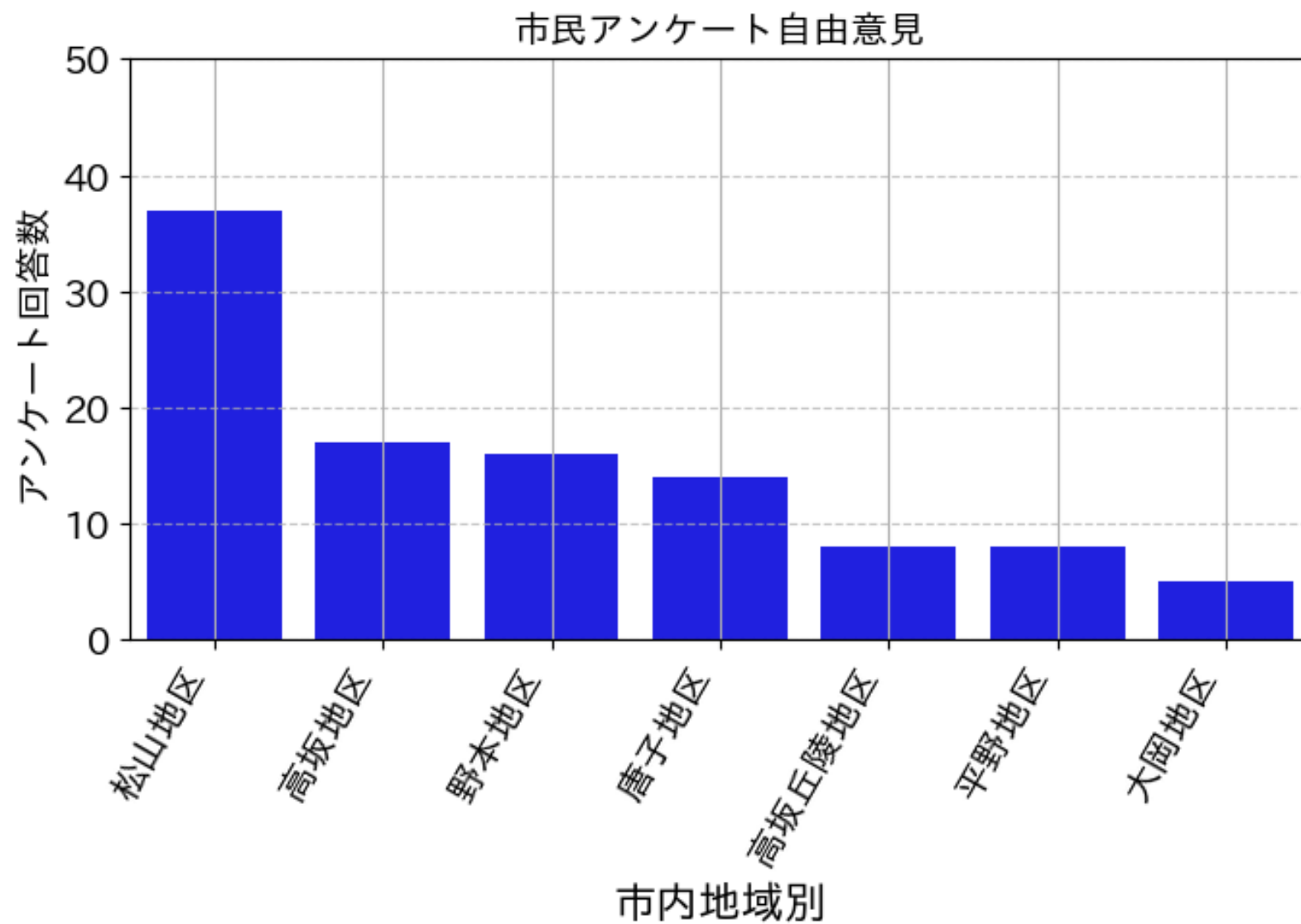
```
plt.ylim(0, 50)
```

```
plt.grid(axis="y", linestyle="--", alpha=0.7)
```

```
plt.tight_layout()
```

```
plt.grid(True)
```

```
plt.show()
```



回答の分野を分ける

```
category = df["分野"].nunique()
print(f"分野は{category}種類です。")
category_list = df["分野"].unique().tolist()
category_dict = df['分野'].value_counts().to_dict()

df_category = pd.DataFrame(list(category_dict.items()),
columns=['分野', 'アンケート回答数'])
display(df_category)
```

	分野	アンケート回答数
0	都市基盤の整備について	19
1	駅周辺や商店街の活性化について	14
2	保健・医療・福祉について	14
3	教育・文化・スポーツについて	14
4	その他	12
5	市政について	9
6	産業について	7
7	防災・防犯・安全について	6
8	環境について	5
9	情報について	3
10	観光について	2

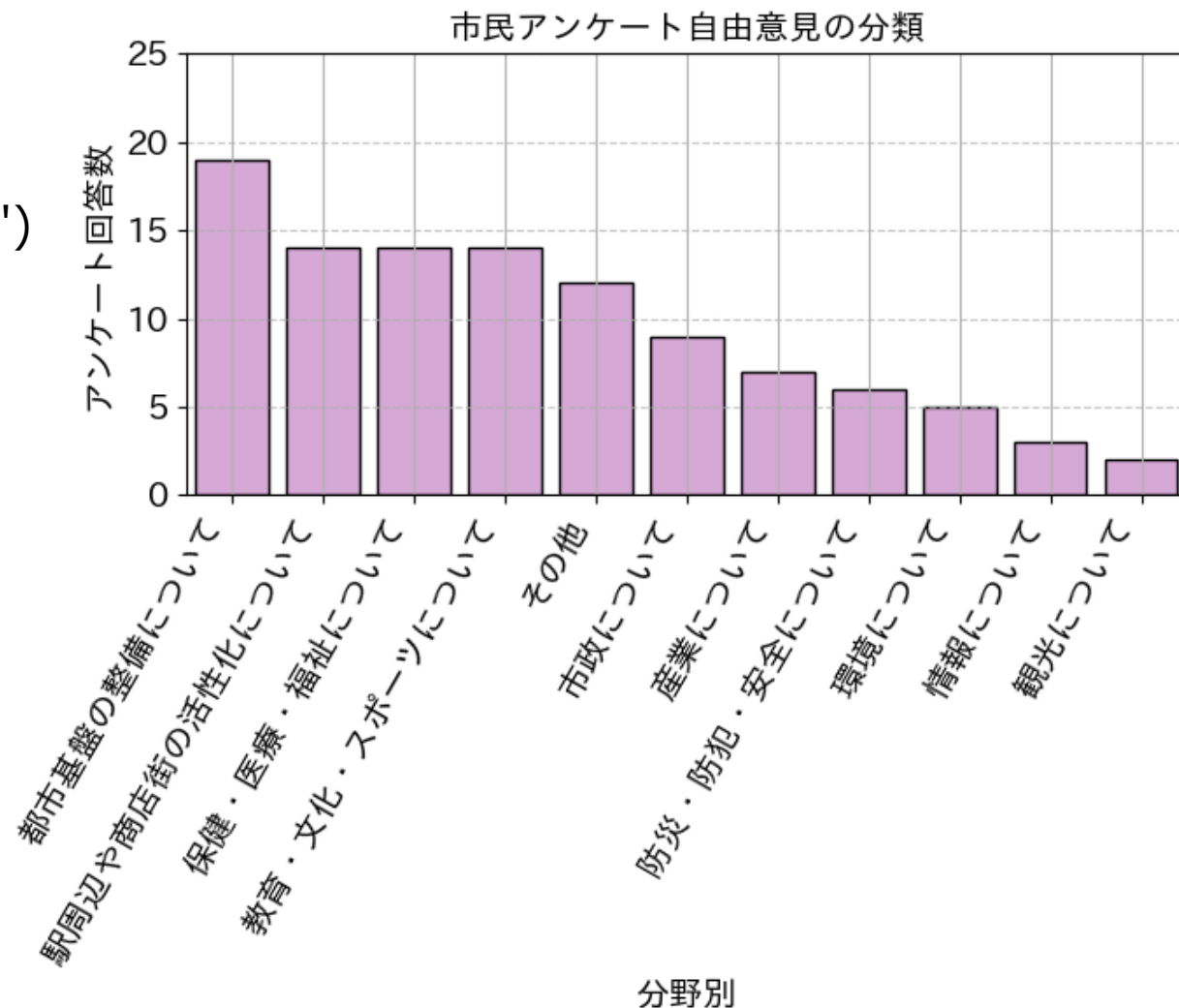
```

plt.figure(figsize=(7, 6))

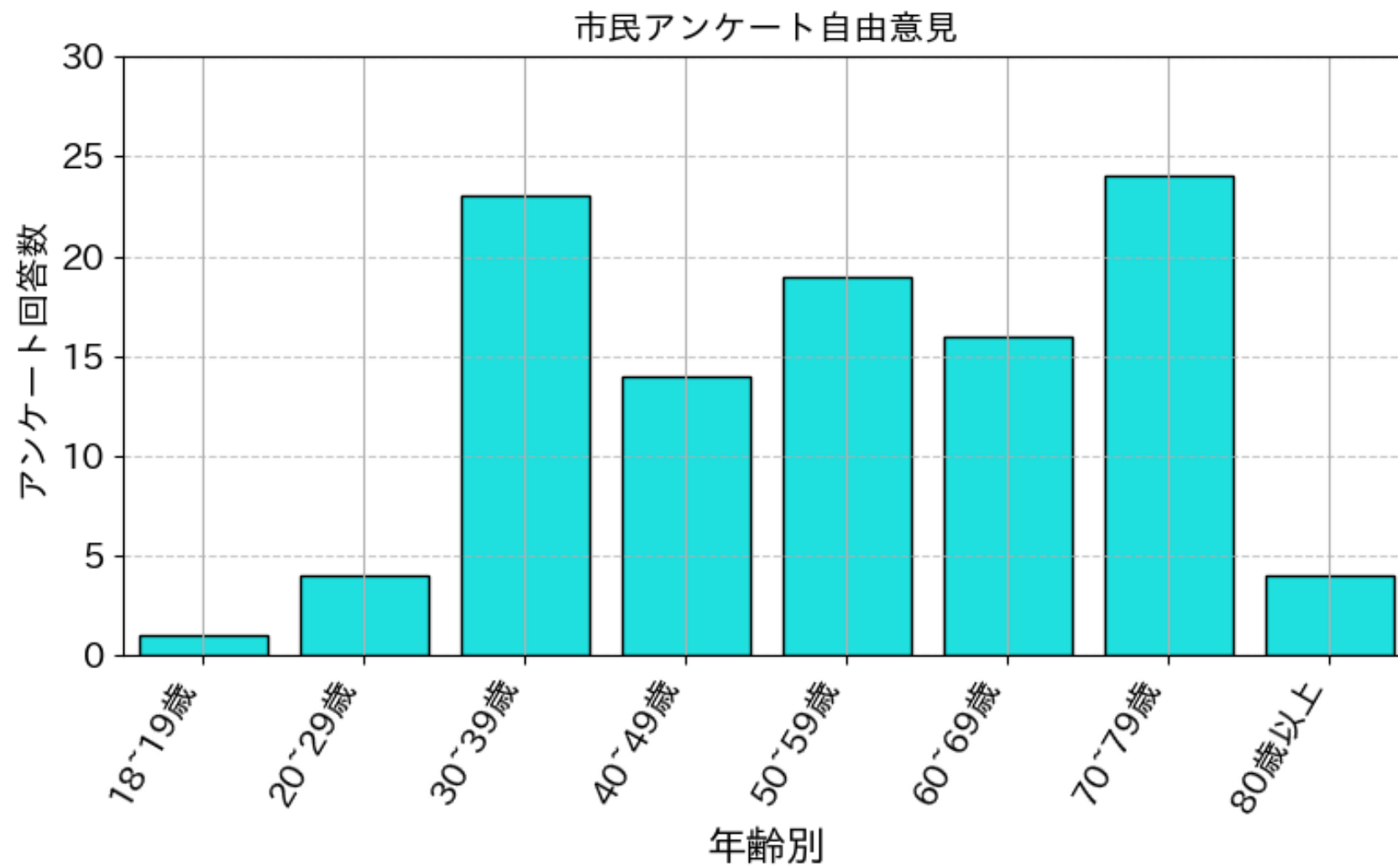
sns.barplot(x="分野", y="アンケート回答数",
data=df_category, color="plum", edgecolor="black")

plt.title("市民アンケート自由意見の分類", fontsize=14)
plt.xticks(rotation=60, ha="right", fontsize=14)
plt.xlabel("分野別", fontsize=14)
plt.ylabel("アンケート回答数", fontsize=14)
plt.yticks(fontsize=14)
plt.ylim(0, 25)
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.grid(True)
plt.show()

```



```
# 年齢
import re
age_dict = df['年齢'].value_counts().to_dict()
# キーの先頭の数値を取り出してソート
sorted_dict = dict(
    sorted(
        age_dict.items(), key=lambda x: int(re.search(r'¥d+', x[0]).group())
    )
)
df_age = pd.DataFrame(list(sorted_dict.items()), columns=['年齢', 'アンケート回答数'])
# グラフ作成
plt.figure(figsize=(8, 5))
sns.barplot(x="年齢", y="アンケート回答数", data=df_age, color="cyan", edgecolor="black")
plt.title("市民アンケート自由意見", fontsize=14)
plt.xticks(rotation=60, ha="right", fontsize=14)
plt.xlabel("年齢別", fontsize=16)
plt.ylabel("アンケート回答数", fontsize=14)
plt.yticks(fontsize=14)
plt.ylim(0, 30)
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.grid(True)
plt.show()
```



R8_東松山市の人口(地域別)

```
# R8_東松山市人口.csvファイル  
csv_files = "/content/mydrive/MyDrive/...../2026/R8_東松山市人口.csv"  
  
# 2. csvファイルを読み込む  
df_population = pd.read_csv(csv_files, encoding="utf-8-sig")  
display(df_population)
```

R8年の東松山市の人口調査(年齢別)

```
# R8_東松山市年齢別人口.csvファイル  
  
csv_files = "/content/mydrive/MyDrive/...../R8_東松山市年齢別人口.csv"  
  
# 2. csvファイルを読み込む  
df_age = pd.read_csv(csv_files, encoding="utf-8-sig")  
display(df_age)
```

R8_東松山市の人口(地域別)棒グラフ

```
# R8_東松山市人口 地区別人口の棒グラフ
```

```
plt.figure(figsize=(10, 6))
```

```
sns.barplot(x="地区", y="合計", data=df_population, color="blue")
```

```
# ラベル・目盛りの設定
```

```
plt.title("R8_東松山市人口 地区別人口", fontsize=16)
```

```
plt.xlabel("地区名", fontsize=16)
```

```
plt.ylabel("人口", fontsize=16)
```

```
plt.xticks(rotation=60, ha="right", fontsize=16)
```

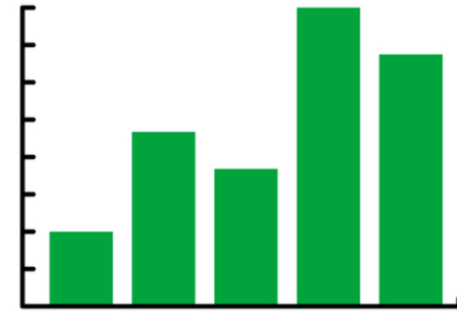
```
plt.yticks(fontsize=16)
```

```
plt.grid(axis="y", linestyle="--", alpha=0.7)
```

```
# レイアウトを自動調整
```

```
plt.tight_layout()
```

```
plt.show()
```



R8年の東松山市の人口調査(年齢別)ヒストグラム

```
x_column = "年齢"  
y_column = "合計"
```

グラフの作成

```
plt.figure(figsize=(12, 6))
```

DataFrameの機能を使って棒グラフを描画

```
df_age.plot.bar(  
    x=x_column,  
    y=y_column,  
    width=1.0,  
    ax=plt.gca(), # 現在の図(figure)に描画  
    color='skyblue',  
    edgecolor='black' # 境界線を黒にして見やすくする  
)
```

軸ラベルとタイトルの設定

```
plt.title('年齢別人口分布 (0歳～105歳)', fontsize=16)  
plt.xlabel('年齢 (歳)', fontsize=16)  
plt.ylabel('人口 (人)', fontsize=16)
```

```
plt.xticks(ticks=range(0, 106, 10), labels=range(0, 106, 10), rotation=0)
```

グリッド線の追加(Y軸のみ)

```
plt.grid(axis='y', linestyle='--', alpha=0.7)  
plt.tight_layout()  
plt.show()
```

集合棒グラフ (Grouped Bar Chart)

```
df_plot = df_grouped.set_index('年齢グループ')
```

2. グラフ作成 kind='bar' で棒グラフを指定

```
df_plot[['男性', '女性', '合計']].plot(  
    kind='bar',  
    color=['royalblue', 'magenta', "brown"],  
    figsize=(12, 6),  
    width=0.8  
)
```

3. グラフ

```
plt.title('2026年 東松山市の年齢階層別の人口分布', fontsize=16)  
plt.xlabel('年齢グループ', fontsize=16)  
plt.ylabel('人口 (人)', fontsize=16)
```

```
plt.xticks(rotation=45, fontsize=16)
```

```
plt.yticks(fontsize=16)
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

レイアウトがはみ出さないように自動調整

```
plt.tight_layout()  
plt.ylim(0, 16000)  
plt.show()
```

ヒストグラムから平均年齢を計算する

```
all_avg_age = (df_age["年齢"] * df_age["合計"]).sum() / df_age["合計"].sum()
print(f"R8_東松山市の平均年齢: {all_avg_age:.1f}歳")

m_avg_age = (df_age["年齢"] * df_age["男性"]).sum() / df_age["男性"].sum()
print(f"R8_東松山市の男性の平均年齢: {m_avg_age:.1f}歳")

f_avg_age = (df_age["年齢"] * df_age["女性"]).sum() / df_age["女性"].sum()
print(f"R8_東松山市の女性の平均年齢: {f_avg_age:.1f}歳")
```

$$\text{平均年齢} = \frac{\sum(\text{年齢} \times \text{その年齢の人口})}{\text{総人口}}$$